CycleGan

March 10, 2026

[ ]:
```
from google.colab import drive
drive.mount("/content/drive")
```

```
Mounted at /content/drive
```

[ ]:
```
from glob import glob
import itertools
import matplotlib.pyplot as plt
import numpy as np
import os
import pandas as pd
import PIL #pillow
from PIL import Image
import random
import shutil
from sklearn.model_selection import GroupKFold #set # of distribution of each␣
↪group
from sklearn.metrics import roc_curve
from sklearn import metrics
import time
from tqdm.notebook import tqdm #progress bars

import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.nn.init as init
from torch.utils.data import Dataset, random_split, DataLoader

import torchvision.models as models
import torchvision.transforms as transforms
```

[ ]:
```
def set_seed(seed):
    random.seed(seed)
    os.environ['PYTHONHASHSEED'] = str(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.backends.cudnn.deterministic = True
```

[ ]:
```
device = 'cuda' if torch.cuda.is_available() else 'cpu'
device
```

[ ]: 'cuda'

[ ]:
```
set_seed(719)
```

[ ]:
```
class ImageDataset(Dataset):
    def __init__(self, monet_dir, photo_dir, size=(256, 256), normalize=True):
        super().__init__()
        self.monet_dir = monet_dir
        self.photo_dir = photo_dir
        self.monet_idx = dict()
        self.photo_idx = dict()
        if normalize:
            self.transform = transforms.Compose([
                transforms.Resize(size),
                transforms.ToTensor(),# /255 -->[0,1]
                transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5)) #␣
 ↪image=(image-mean)/std --> [-1,1]
            ])
        else:
            self.transform = transforms.Compose([
                transforms.Resize(size),
                transforms.ToTensor()
            ])
        for i, fl in enumerate(os.listdir(self.monet_dir)):
            self.monet_idx[i] = fl
        for i, fl in enumerate(os.listdir(self.photo_dir)):
            self.photo_idx[i] = fl

    def __getitem__(self, idx):
        rand_idx = int(np.random.uniform(0, len(self.monet_idx.keys())))
        photo_path = os.path.join(self.photo_dir, self.photo_idx[rand_idx])
        monet_path = os.path.join(self.monet_dir, self.monet_idx[idx])
        photo_img = Image.open(photo_path)
        photo_img = self.transform(photo_img)
        monet_img = Image.open(monet_path)
        monet_img = self.transform(monet_img)
        return photo_img, monet_img

    def __len__(self):
        return min(len(self.monet_idx.keys()), len(self.photo_idx.keys()))
```

[ ]:
```
img_ds = ImageDataset('/content/drive/My Drive/gan/gan-getting-started/
 ↪monet_jpg/', '/content/drive/My Drive/gan/gan-getting-started/photo_jpg/')
```

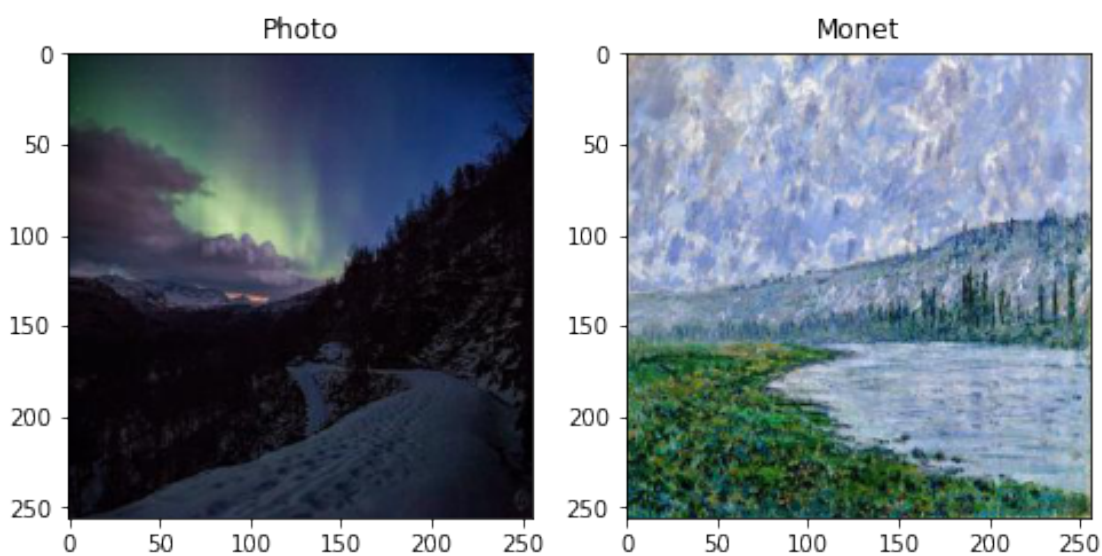
[ ]:
```
img_dl = DataLoader(img_ds, batch_size=1, pin_memory=True)
```

[ ]:
```
photo_img, monet_img = next(iter(img_dl)) #li=[1,2,3]; it=iter(li); next(it)-->1
```

[ ]:
```
def unnorm(img, mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5]):
    for t, m, s in zip(img, mean, std):
        t.mul_(s).add_(s)

    return img
```

[ ]:
```
f = plt.figure(figsize=(8, 8))

f.add_subplot(1, 2, 1)
plt.title('Photo')
photo_img = unnorm(photo_img)
plt.imshow(photo_img[0].permute(1, 2, 0)) #256*256*3

f.add_subplot(1, 2, 2)
plt.title('Monet')
monet_img = unnorm(monet_img)
plt.imshow(monet_img[0].permute(1, 2, 0))
```

[ ]:
```
<matplotlib.image.AxesImage at 0x7f4f3688ee10>
```



Save and load

[ ]:
```
def load_checkpoint(ckpt_path, map_location=None):
    ckpt = torch.load(ckpt_path, map_location=map_location)
    print(' [*] Loading checkpoint from %s succeed!' % ckpt_path)
```

```
    return ckpt
```

[ ]:
```
def save_checkpoint(state, save_path):
    torch.save(state, save_path)
```

Model

[ ]:
```
#  make abstract images' spatial dimensions equal to the input image
def Upsample(in_ch, out_ch, use_dropout=True, dropout_ratio=0.5):
    if use_dropout:
        return nn.Sequential(
            nn.ConvTranspose2d(in_ch, out_ch, 3, stride=2, padding=1,␣
 ↪output_padding=1),#stride=1,padding;stride>1,padding+interpolation
            nn.InstanceNorm2d(out_ch),
            nn.Dropout(dropout_ratio),
            nn.GELU()
        )
    else:
        return nn.Sequential(
            nn.ConvTranspose2d(in_ch, out_ch, 3, stride=2, padding=1,␣
 ↪output_padding=1),
            nn.InstanceNorm2d(out_ch),
            nn.GELU()
        )
```

[ ]:
```
def Convlayer(in_ch, out_ch, kernel_size=3, stride=2, use_leaky=True,␣
 ↪use_inst_norm=True, use_pad=True):
    if use_pad:
        conv = nn.Conv2d(in_ch, out_ch, kernel_size, stride, 1, bias=True)
    else:
        conv = nn.Conv2d(in_ch, out_ch, kernel_size, stride, 0, bias=True)

    if use_leaky:
        actv = nn.LeakyReLU(negative_slope=0.2, inplace=True)
    else:
        actv = nn.GELU()

    if use_inst_norm:
        norm = nn.InstanceNorm2d(out_ch)
    else:
        norm = nn.BatchNorm2d(out_ch) # normalization by batch: prevent␣
 ↪exploding & vanishing gradient

    return nn.Sequential(
        conv,
        norm,
        actv
```

```
    )
```

[ ]:
```
class Resblock(nn.Module):
    def __init__(self, in_features, use_dropout=True, dropout_ratio=0.5):
        super().__init__()
        layers = list()
        layers.append(nn.ReflectionPad2d(1))
        layers.append(Convlayer(in_features, in_features, 3, 1, False,␣
 ↪use_pad=False))
        layers.append(nn.Dropout(dropout_ratio))
        layers.append(nn.ReflectionPad2d(1))
        layers.append(nn.Conv2d(in_features, in_features, 3, 1, padding=0,␣
 ↪bias=True))
        layers.append(nn.InstanceNorm2d(in_features))
        self.res = nn.Sequential(*layers)#*Receives non-keyword parameters in␣
 ↪tuples

    def forward(self, x):
        return x + self.res(x)
```

[ ]:
```
class Generator(nn.Module):
    def __init__(self, in_ch, out_ch, num_res_blocks=6):
        super().__init__()
        model = list()
        model.append(nn.ReflectionPad2d(3))
        model.append(Convlayer(in_ch, 64, 7, 1, False, True, False))
        model.append(Convlayer(64, 128, 3, 2, False))
        model.append(Convlayer(128, 256, 3, 2, False))
        for _ in range(num_res_blocks):
            model.append(Resblock(256))
        model.append(Upsample(256, 128))
        model.append(Upsample(128, 64))
        model.append(nn.ReflectionPad2d(3))
        model.append(nn.Conv2d(64, out_ch, kernel_size=7, padding=0))
        model.append(nn.Tanh())

        self.gen = nn.Sequential(*model)

    def forward(self, x):
        return self.gen(x)
```

[ ]:
```
class Discriminator(nn.Module):
    def __init__(self, in_ch, num_layers=4):
        super().__init__()
        model = list()
        model.append(nn.Conv2d(in_ch, 64, 4, stride=2, padding=1))
        model.append(nn.LeakyReLU(0.2, inplace=True))
```

```
        for i in range(1, num_layers):
            in_chs = 64 * 2**(i-1)
            out_chs = in_chs * 2
            if i == num_layers -1:
                model.append(Convlayer(in_chs, out_chs, 4, 1))
            else:
                model.append(Convlayer(in_chs, out_chs, 4, 2))
        model.append(nn.Conv2d(512, 1, kernel_size=4, stride=1, padding=1))
        self.disc = nn.Sequential(*model)# "*" deconstruction of tuple: a =␣
↪(1,2,3) def f(b,c,d): pass f(*a) # b = 1, c = 2, d = 3

    def forward(self, x):
        return self.disc(x)
```

[ ]:
```
def init_weights(net, init_type='normal', gain=0.02):
    def init_func(m):
        classname = m.__class__.__name__
        if hasattr(m, 'weight') and (classname.find('Conv') != -1 ortorch.nn.
↪init classname.find('Linear') != -1):
            init.normal_(m.weight.data, 0.0, gain) # torch.nn.init
            if hasattr(m, 'bias') and m.bias is not None:
                init.constant_(m.bias.data, 0.0)
        elif classname.find('BatchNorm2d') != -1:
            init.normal_(m.weight.data, 1.0, gain)
            init.constant_(m.bias.data, 0.0)
    net.apply(init_func)
```

Addtional class and function

[ ]:
```
def update_req_grad(models, requires_grad=True):
    for model in models:
        for param in model.parameters():
            param.requires_grad = requires_grad
```

[ ]:
```
# https://arxiv.org/pdf/1612.07828.pdf
# Save 50 generated fake imgs and sample through them
# to feed discriminators to avoid large oscillations
# from iterations to iterations.
class sample_fake(object):
    def __init__(self, max_imgs=50):
        self.max_imgs = max_imgs
        self.cur_img = 0
        self.imgs = list()

    def __call__(self, imgs):
        ret = list()
        for img in imgs:
```

```
            if self.cur_img < self.max_imgs:
                self.imgs.append(img)
                ret.append(img)
                self.cur_img += 1
            else:
                if np.random.ranf() > 0.5:
                    idx = np.random.randint(0, self.max_imgs)
                    ret.append(self.imgs[idx])
                    self.imgs[idx] = img
                else:
                    ret.append(img)
        return ret
```

[ ]:
```
class lr_sched():
    def __init__(self, decay_epochs=100, total_epochs=200):
        self.decay_epochs = decay_epochs
        self.total_epochs = total_epochs

    def step(self, epoch_num):
        if epoch_num <= self.decay_epochs:
            return 1.0
        else:
            fract = (epoch_num - self.decay_epochs)  / (self.total_epochs -␣
 ↪self.decay_epochs)
            return 1.0 - fract
```

[ ]:
```
class AvgStats(object):
    def __init__(self):
        self.reset()

    def reset(self):
        self.losses =[]
        self.its = []

    def append(self, loss, it):
        self.losses.append(loss)
        self.its.append(it)
```

GAN Class

[ ]:
```
class CycleGAN(object):
    def __init__(self, in_ch, out_ch, epochs, device, start_lr=2e-4, lmbda=10,␣
 ↪idt_coef=0.5, decay_epoch=0):
        self.epochs = epochs
        self.decay_epoch = decay_epoch if decay_epoch > 0 else int(self.epochs/
 ↪2)
        self.lmbda = lmbda
```

```
        self.idt_coef = idt_coef
        self.device = device
        self.gen_mtp = Generator(in_ch, out_ch)#monet to photo
        self.gen_ptm = Generator(in_ch, out_ch)# photo to monet
        self.desc_m = Discriminator(in_ch)
        self.desc_p = Discriminator(in_ch)
        self.init_models()
        self.mse_loss = nn.MSELoss()
        self.l1_loss = nn.L1Loss() #MAE
        self.adam_gen = torch.optim.Adam(itertools.chain(self.gen_mtp.
↪parameters(), self.gen_ptm.parameters()),
                                         lr = start_lr, betas=(0.5, 0.999))
        self.adam_desc = torch.optim.Adam(itertools.chain(self.desc_m.
↪parameters(), self.desc_p.parameters()),
                                          lr=start_lr, betas=(0.5, 0.999))
        self.sample_monet = sample_fake()
        self.sample_photo = sample_fake()
        gen_lr = lr_sched(self.decay_epoch, self.epochs)
        desc_lr = lr_sched(self.decay_epoch, self.epochs)
        self.gen_lr_sched = torch.optim.lr_scheduler.LambdaLR(self.adam_gen,␣
↪gen_lr.step)
        self.desc_lr_sched = torch.optim.lr_scheduler.LambdaLR(self.adam_desc,␣
↪desc_lr.step)
        self.gen_stats = AvgStats()
        self.desc_stats = AvgStats()

    def init_models(self):
        init_weights(self.gen_mtp)
        init_weights(self.gen_ptm)
        init_weights(self.desc_m)
        init_weights(self.desc_p)
        self.gen_mtp = self.gen_mtp.to(self.device)
        self.gen_ptm = self.gen_ptm.to(self.device)
        self.desc_m = self.desc_m.to(self.device)
        self.desc_p = self.desc_p.to(self.device)

    def train(self, photo_dl):
        for epoch in range(self.epochs):
            start_time = time.time()
            avg_gen_loss = 0.0
            avg_desc_loss = 0.0
            t = tqdm(photo_dl, leave=False, total=photo_dl.__len__())
            for i, (photo_real, monet_real) in enumerate(t):
                photo_img, monet_img = photo_real.to(device), monet_real.
↪to(device)
                update_req_grad([self.desc_m, self.desc_p], False)
                self.adam_gen.zero_grad()
```

```
                # Forward pass through generator
                fake_photo = self.gen_mtp(monet_img)
                fake_monet = self.gen_ptm(photo_img)

                cycl_monet = self.gen_ptm(fake_photo)
                cycl_photo = self.gen_mtp(fake_monet)

                id_monet = self.gen_ptm(monet_img)
                id_photo = self.gen_mtp(photo_img)

                # generator losses - identity, Adversarial, cycle consistency
                idt_loss_monet = self.l1_loss(id_monet, monet_img) * self.lmbda␣
↪* self.idt_coef
                idt_loss_photo = self.l1_loss(id_photo, photo_img) * self.lmbda␣
↪* self.idt_coef

                cycle_loss_monet = self.l1_loss(cycl_monet, monet_img) * self.
↪lmbda
                cycle_loss_photo = self.l1_loss(cycl_photo, photo_img) * self.
↪lmbda

                monet_desc = self.desc_m(fake_monet)
                photo_desc = self.desc_p(fake_photo)

                real = torch.ones(monet_desc.size()).to(self.device)

                adv_loss_monet = self.mse_loss(monet_desc, real)
                adv_loss_photo = self.mse_loss(photo_desc, real)

                # total generator loss
                total_gen_loss = cycle_loss_monet + adv_loss_monet\
                               + cycle_loss_photo + adv_loss_photo\
                               + idt_loss_monet + idt_loss_photo

                avg_gen_loss += total_gen_loss.item()

                # backward pass
                total_gen_loss.backward()
                self.adam_gen.step()

                # Forward pass through Descriminator
                update_req_grad([self.desc_m, self.desc_p], True)
                self.adam_desc.zero_grad()

                fake_monet = self.sample_monet([fake_monet.cpu().data.
↪numpy()])[0]
```

```
                fake_photo = self.sample_photo([fake_photo.cpu().data.
↪numpy()])[0]
                fake_monet = torch.tensor(fake_monet).to(self.device)
                fake_photo = torch.tensor(fake_photo).to(self.device)

                monet_desc_real = self.desc_m(monet_img)
                monet_desc_fake = self.desc_m(fake_monet)
                photo_desc_real = self.desc_p(photo_img)
                photo_desc_fake = self.desc_p(fake_photo)

                real = torch.ones(monet_desc_real.size()).to(self.device)
                fake = torch.zeros(monet_desc_fake.size()).to(self.device)

                # Descriminator losses
                # --------------------
                monet_desc_real_loss = self.mse_loss(monet_desc_real, real)
                monet_desc_fake_loss = self.mse_loss(monet_desc_fake, fake)
                photo_desc_real_loss = self.mse_loss(photo_desc_real, real)
                photo_desc_fake_loss = self.mse_loss(photo_desc_fake, fake)

                monet_desc_loss = (monet_desc_real_loss + monet_desc_fake_loss)␣
↪/ 2
                photo_desc_loss = (photo_desc_real_loss + photo_desc_fake_loss)␣
↪/ 2
                total_desc_loss = monet_desc_loss + photo_desc_loss
                avg_desc_loss += total_desc_loss.item()

                # Backward
                monet_desc_loss.backward()
                photo_desc_loss.backward()
                self.adam_desc.step()

                t.set_postfix(gen_loss=total_gen_loss.item(),␣
↪desc_loss=total_desc_loss.item())

            save_dict = {
                'epoch': epoch+1,
                'gen_mtp': gan.gen_mtp.state_dict(),
                'gen_ptm': gan.gen_ptm.state_dict(),
                'desc_m': gan.desc_m.state_dict(),
                'desc_p': gan.desc_p.state_dict(),
                'optimizer_gen': gan.adam_gen.state_dict(),
                'optimizer_desc': gan.adam_desc.state_dict()
            }
            save_checkpoint(save_dict, 'current.ckpt')

            avg_gen_loss /= photo_dl.__len__()
```

```
            avg_desc_loss /= photo_dl.__len__()
            time_req = time.time() - start_time

            self.gen_stats.append(avg_gen_loss, time_req)
            self.desc_stats.append(avg_desc_loss, time_req)

            print("Epoch: (%d) | Generator Loss:%f | Discriminator Loss:%f" %
                                                (epoch+1, avg_gen_loss,␣
 ↪avg_desc_loss))

            self.gen_lr_sched.step()
            self.desc_lr_sched.step()
```

[ ]:
```
gan = CycleGAN(3, 3, 50, device)
```

[ ]:
```
# Save before train
save_dict = {
    'epoch': 0,
    'gen_mtp': gan.gen_mtp.state_dict(),
    'gen_ptm': gan.gen_ptm.state_dict(),
    'desc_m': gan.desc_m.state_dict(),
    'desc_p': gan.desc_p.state_dict(),
    'optimizer_gen': gan.adam_gen.state_dict(),
    'optimizer_desc': gan.adam_desc.state_dict()
}
```

[ ]:
```
save_checkpoint(save_dict, 'init.ckpt')
```

[ ]:
```
gan.train(img_dl)
```

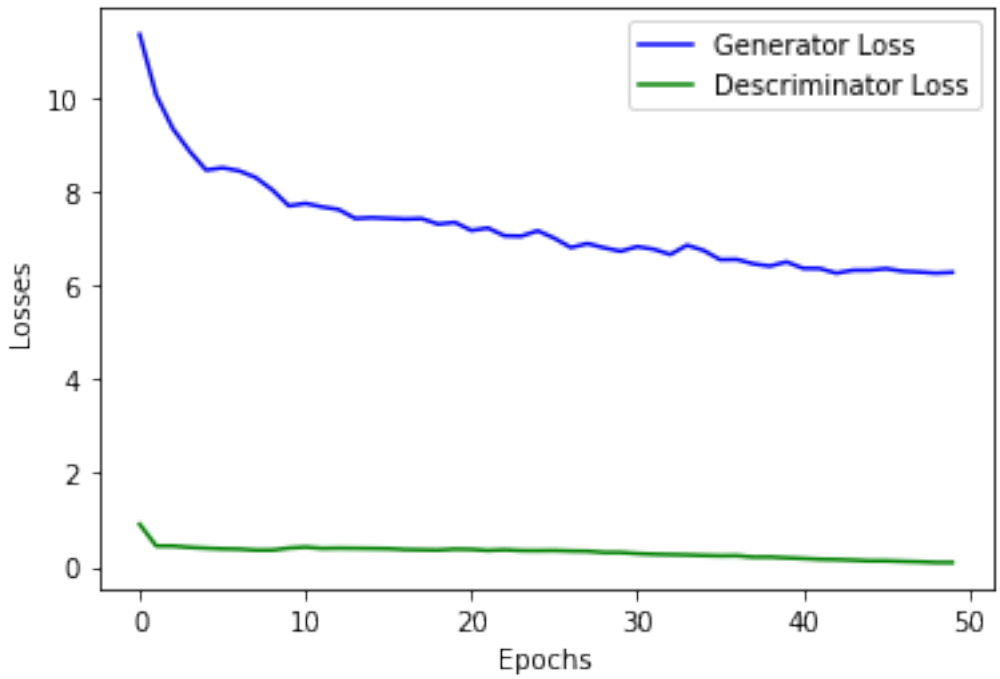
```
  0%|          | 0/300 [00:00<?, ?it/s]

Epoch: (1) | Generator Loss:11.348638 | Discriminator Loss:0.909477

  0%|          | 0/300 [00:00<?, ?it/s]

Epoch: (2) | Generator Loss:10.051679 | Discriminator Loss:0.445002

  0%|          | 0/300 [00:00<?, ?it/s]

Epoch: (3) | Generator Loss:9.340576 | Discriminator Loss:0.444083

  0%|          | 0/300 [00:00<?, ?it/s]

Epoch: (4) | Generator Loss:8.864758 | Discriminator Loss:0.416076

  0%|          | 0/300 [00:00<?, ?it/s]

Epoch: (5) | Generator Loss:8.456542 | Discriminator Loss:0.398236

  0%|          | 0/300 [00:00<?, ?it/s]

Epoch: (6) | Generator Loss:8.505346 | Discriminator Loss:0.384971
```

```
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (7) | Generator Loss:8.440864 | Discriminator Loss:0.378707
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (8) | Generator Loss:8.298115 | Discriminator Loss:0.356256
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (9) | Generator Loss:8.033428 | Discriminator Loss:0.357091
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (10) | Generator Loss:7.693084 | Discriminator Loss:0.400836
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (11) | Generator Loss:7.747003 | Discriminator Loss:0.420805
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (12) | Generator Loss:7.671476 | Discriminator Loss:0.393087
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (13) | Generator Loss:7.616446 | Discriminator Loss:0.399028
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (14) | Generator Loss:7.425473 | Discriminator Loss:0.395391
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (15) | Generator Loss:7.441695 | Discriminator Loss:0.391634
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (16) | Generator Loss:7.427073 | Discriminator Loss:0.387710
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (17) | Generator Loss:7.412244 | Discriminator Loss:0.368330
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (18) | Generator Loss:7.422754 | Discriminator Loss:0.362685
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (19) | Generator Loss:7.304822 | Discriminator Loss:0.357115
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (20) | Generator Loss:7.343094 | Discriminator Loss:0.377318
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (21) | Generator Loss:7.169661 | Discriminator Loss:0.372200
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (22) | Generator Loss:7.221228 | Discriminator Loss:0.348955
```

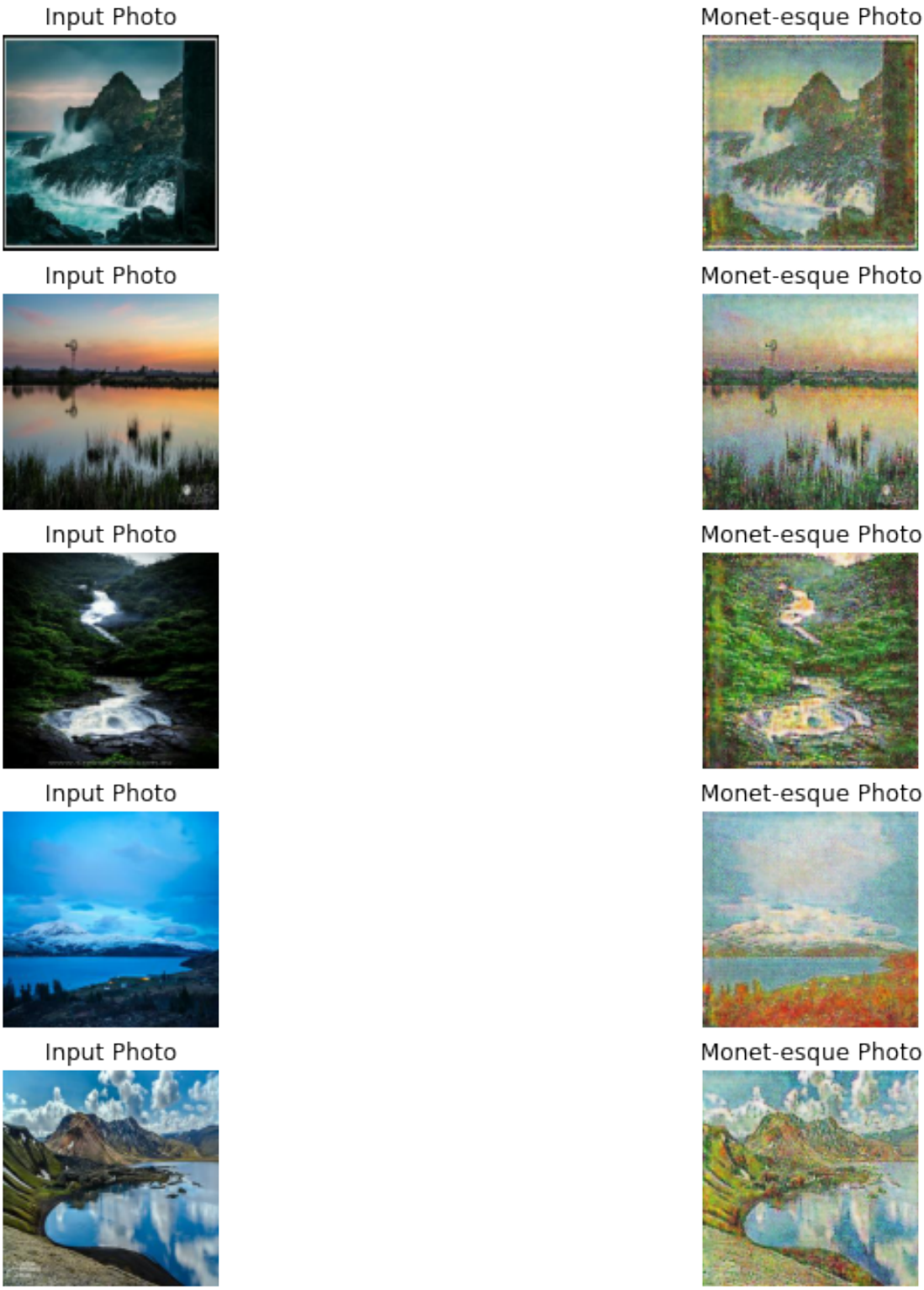
```
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (23) | Generator Loss:7.056146 | Discriminator Loss:0.363064
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (24) | Generator Loss:7.047400 | Discriminator Loss:0.346744
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (25) | Generator Loss:7.163003 | Discriminator Loss:0.345037
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (26) | Generator Loss:7.005265 | Discriminator Loss:0.347915
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (27) | Generator Loss:6.805677 | Discriminator Loss:0.337351
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (28) | Generator Loss:6.891090 | Discriminator Loss:0.333025
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (29) | Generator Loss:6.800094 | Discriminator Loss:0.307280
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (30) | Generator Loss:6.727389 | Discriminator Loss:0.306498
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (31) | Generator Loss:6.825739 | Discriminator Loss:0.282734
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (32) | Generator Loss:6.768564 | Discriminator Loss:0.268555
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (33) | Generator Loss:6.658543 | Discriminator Loss:0.261001
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (34) | Generator Loss:6.855806 | Discriminator Loss:0.254164
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (35) | Generator Loss:6.744993 | Discriminator Loss:0.243576
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (36) | Generator Loss:6.546991 | Discriminator Loss:0.235185
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (37) | Generator Loss:6.552465 | Discriminator Loss:0.238198
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (38) | Generator Loss:6.459425 | Discriminator Loss:0.208031
```

```
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (39) | Generator Loss:6.409757 | Discriminator Loss:0.209359
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (40) | Generator Loss:6.501002 | Discriminator Loss:0.191630
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (41) | Generator Loss:6.361946 | Discriminator Loss:0.177688
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (42) | Generator Loss:6.358980 | Discriminator Loss:0.161153
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (43) | Generator Loss:6.259104 | Discriminator Loss:0.154690
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (44) | Generator Loss:6.321881 | Discriminator Loss:0.143396
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (45) | Generator Loss:6.322801 | Discriminator Loss:0.127848
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (46) | Generator Loss:6.355012 | Discriminator Loss:0.128167
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (47) | Generator Loss:6.300984 | Discriminator Loss:0.116250
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (48) | Generator Loss:6.283544 | Discriminator Loss:0.107674
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (49) | Generator Loss:6.259555 | Discriminator Loss:0.092845
  0%|          | 0/300 [00:00<?, ?it/s]
Epoch: (50) | Generator Loss:6.276937 | Discriminator Loss:0.092324
```

[39]:
```
plt.xlabel("Epochs")
plt.ylabel("Losses")
plt.plot(gan.gen_stats.losses, 'b', label='Generator Loss')
plt.plot(gan.desc_stats.losses, 'g', label='Descriminator Loss')
plt.legend()
plt.show()
```



[ ]:
```
_, ax = plt.subplots(5, 2, figsize=(12, 12))
for i in range(5):
    photo_img, _ = next(iter(img_dl))
    pred_monet = gan.gen_ptm(photo_img.to(device)).cpu().detach()
    photo_img = unnorm(photo_img)
    pred_monet = unnorm(pred_monet)

    ax[i, 0].imshow(photo_img[0].permute(1, 2, 0))
    ax[i, 1].imshow(pred_monet[0].permute(1, 2, 0))
    ax[i, 0].set_title("Input Photo")
    ax[i, 1].set_title("Monet-esque Photo")
    ax[i, 0].axis("off")
    ax[i, 1].axis("off")
plt.show()
```



Generate all images

[ ]:
```
class PhotoDataset(Dataset):
    def __init__(self, photo_dir, size=(256, 256), normalize=True):
        super().__init__()
        self.photo_dir = photo_dir
        self.photo_idx = dict()
        if normalize:
            self.transform = transforms.Compose([
                transforms.Resize(size),
                transforms.ToTensor(),
                transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
            ])
        else:
            self.transform = transforms.Compose([
                transforms.Resize(size),
                transforms.ToTensor()
            ])
        for i, fl in enumerate(os.listdir(self.photo_dir)):
            self.photo_idx[i] = fl

    def __getitem__(self, idx):
        photo_path = os.path.join(self.photo_dir, self.photo_idx[idx])
        photo_img = Image.open(photo_path)
        photo_img = self.transform(photo_img)
        return photo_img

    def __len__(self):
        return len(self.photo_idx.keys())
```

[ ]:
```
ph_ds = PhotoDataset('/content/drive/My Drive/gan/gan-getting-started/photo_jpg/
↪')
```

[ ]:
```
ph_dl = DataLoader(ph_ds, batch_size=1, pin_memory=True)
```

[ ]:
```
trans = transforms.ToPILImage()
```

[ ]:
```
t = tqdm(ph_dl, leave=False, total=ph_dl.__len__())
for i, photo in enumerate(t):
    with torch.no_grad():
        pred_monet = gan.gen_ptm(photo.to(device)).cpu().detach()
    pred_monet = unnorm(pred_monet)
    img = trans(pred_monet[0]).convert("RGB")
    img.save("/content/drive/My Drive/gan/images2/" + str(i+1) + ".jpg")
```

Reference

https://www.kaggle.com/nachiket273/cyclegan-pytorch

https://www.kaggle.com/shivansh002/gentle-introduction-to-gan